

Project 3 Report: 2048 Game

CMPE 230 Systems Programming - Spring 2026

Ahmet Cem Çalık, 2022400171

June 7, 2026

1 Project Overview

The source code provides an executable that launches a 2048 app that runs on a single window with all the information and buttons laid out on the main page. User can choose the desired game modes (Normal, Unlimited and Hard) with push buttons below the title and reset the game or undo the last move(s) with the push buttons on the top left of the grid or with their respective keys.

Game starts with the Normal mode being the default selected game mode. Initially two tiles will be selected randomly and will contain a '2' inside them. Player will slide the grid with WASD or the arrow keys to merge the tiles. If the desired value with 2048 as the default value is reached, user will be notified with the "You Win!" overlay, at this point they can decide to start a new game or continue in Unlimited mode. If there are no valid moves left, user will again be notified with an overlay at which point they have to restart to continue doing anything.

2 User Interface Design

UI Components

The application contains a single main window with the game board integrated into it. Main window contains all the push buttons, titles, score trackers and the timer that is only visible in Hard mode along with the main gameplay grid.

- **QVBoxLayout:** The main page layout is initially vertically configured, with title and score labels sitting at the topmost layer (the header), buttons below that and gameplay grid at the bottom.
- **QHBoxLayout:** Layers of vertical layout are divided horizontally to house different label, such as the title being the leftmost and scores being the rightmost elements of the header.
- **QGridLayout:** Used for the gameplay grid to organize the NxM matrix.

Within the main page, title and scores are displayed as **QLabel** to provide the user with visual feedback.

The game mode selection buttons and restart-undo options are displayed as **QPushButton** to provide interactivity. Latter two are also provided with **QKeySequence** to provide the user with the option to interact with these buttons using the keyboard.

The gameboard is a custom **QWidget** that displays the container for the tiles as **QGridLayout** and the tiles themselves as **QLabel**. Gameboard also implements **QTimer** to provide the user with a timer that displays the time window to complete the next move.

The game ending states are displayed to user with **QMessageBox**. Winning overlay also includes **QPushButtons** within the message box to provide the user with options of continuation, wheres ending overlay is a simple **Information** message box.

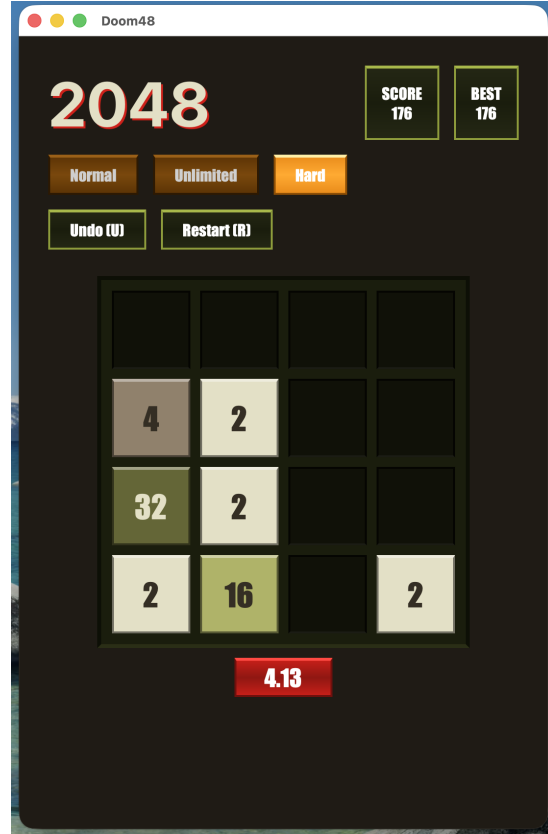


Figure 1: Screenshot of the app.

Connection

Connections between the UI and the functionalities are supported via the SIGNAL and SLOT mechanisms of Qt. When UI buttons are clicked, this SIGNAL is sent to the SLOTS via the lambda functions of C++, such as `gameBoard->undo()` with the click of the U button for an undo on the grid move. The opposite direction of the mechanism, logic to UI, sends custom signals to UI for the changes, such as `emit scoreUpdated(newScore)` to broadcast the score into the score label on the main page. Main window always listens for this incoming signal.

3 Game State Representation

Internally, the game board is completely decoupled from the UI. Logically, it is a 2D vector of integers using **QVector**. Although not shown in the UI, empty cells are denoted as 0 in the code. During rendering, UI checks for the values and if the tile value equals 0 it renders it as an empty string ("").

Merging

Merging is handled by extracting rows or columns into 1D arrays, compressing out the empty spaces, and doubling adjacent matching values. Merging is explained in detail in following sections.

Score Tracking

- The current score is tracked via a simple int `currentScore` variable. It is heavily tied to the merge logic mentioned above. Whenever two tiles merge, the resulting new value is added to the score. For example, merging two 16s adds 32 to `currentScore`. The logic engine then fires off the emit `scoreUpdated(currentScore)` signal to tell the UI to redraw the label.
- The best score is tracked by an int `bestScore` variable. Every time `currentScore` increases, the game runs a quick check: if (`currentScore > bestScore`). If true, `bestScore` is overwritten, and a `bestScoreUpdate` signal is sent to the UI.

Undo System

Undo system relies on a LIFO system and therefore requires a stack. This stack is implemented with a `QState` that holds elements of the struct `GameState`, which holds the 2D array and the score.

Right before the tile-sliding algorithm executes, the game takes a "snapshot" of the current board and score using a function like `captureCurrentState()`. If the move after this snapshot is valid, snapshot is pushed onto the stack. If not, it gets discarded.

If the player wants to undo their move, a value from the stack is popped and the board is overwritten with the data from the popped `GameState`. Then the UI makes the necessary changes.

4 Event-Driven Architecture and Game Logic

4.0.1 Keyboard and Button Events

We bypassed standard signals and directly overrode the `keyPressEvent(QKeyEvent *event)` function inside `GameBoard`. By directly intercepting the event, the board instantly reads the arrow key and triggers the corresponding math loop (e.g., `slideUp()`). The function only captures undo history, spawns new tiles, and updates the score if the math engine reports that `boardChanged == true`.

4.1 Signals, Slots, and Connections

In this application, Qt's signal and slot mechanism serves as the communication bridge between the view and the logic. Event-driven architecture ensures that UI, timers, and

game logic remain decoupled. When an event occurs, an object emits a signal, which is immediately caught by a connected slot (or an inline C++ lambda function) to execute the corresponding logic.

Signal Emitted	Connected Slot / Lambda	Purpose
QPushButton::clicked()	[=]() { gameBoard->undo(); }	Reverts board to the previous state.
QPushButton::clicked()	[=]() { gameBoard->setGameMode(); }	Toggles game rules and adjusts timers.
QTimer::timeout()	GameBoard::hardModeTimeout()	Forces a random move if time expires.
QTimer::timeout()	GameBoard::updateTimerUI()	Updates the digital clock label smoothly.
GameBoard::scoreUpdated()	[=](int) { scoreLabel->setText(); }	Updates the visual score counter.

Table 1: Main Qt signal-slot connections used to drive the application’s logic.

These connections are established strictly during initialization within the class constructors (`MainWindow::MainWindow` and `GameBoard::GameBoard`).

- **User Input Connections:** The `QPushButton` widgets emit `clicked()` signals when interacted with. The `MainWindow` captures these via inline lambda functions and commands the `GameBoard` to alter its internal state (e.g., executing an undo operation, restarting, or switching to Hard Mode).
- **Timer Connections:** The `hardModeTimer` and `uiTimer` objects emit `timeout()` signals continuously in the background. The `GameBoard` receives these to either trigger strict game logic (forcing a random matrix slide and evaluating win/loss conditions) or to update the digital UI clock.
- **Custom Logic Connections:** When the math engine successfully merges tiles or changes states, the `GameBoard` emits custom signals such as `scoreUpdated(int)` and `modeChanged(GameMode)`. The `MainWindow` receives these signals and updates the visual UI components (`QLabel` text and button background colors) ensuring the visual layer perfectly reflects the mathematical reality without directly interfering with the logic engine.

4.2 Move and Merge Logic

After the line selection is made (all lines get selected iteratively) for the chosen direction to slide and the selected line is passed to the merger function `mergeLine()`. This function takes this 1D line and gets rid of all the zeros and merges adjacent tiles by comparing them and doubling the first one and making the second one zero if they are equal. Then the newly created zeros are deleted from the list for the second time. After the merging process, the resulting number is compared against the winning constant K to see if the winning condition is met. Then, the resulting line is compared with the original to see if the move resulted in a change, whether its a merge or just tile displacement.

4.3 Timer-Based Events

The Hard Mode mechanic is driven by a background `QTimer` configured to enforce a five-second action window. If the timer reaches zero without user input, it emits a timeout

signal that triggers a specialized fallback function. This function randomly selects a slide direction and tests it against the grid; if the move is physically blocked, it iterates to the next direction until a mathematically valid slide is successfully executed. Following this forced move, a new tile is spawned, and the background timer automatically restarts its countdown. To seamlessly integrate player agency, any manual, valid keystroke made by the user will also instantly reset this five-second clock, ensuring the pressure mechanic continuously adapts to active gameplay.

At this point, no check for valid moves is necessary since that check already happens before the execution of forced slides, so we know for certain that at least one direction will result in a successful slide.

5 Code Structure

In the source folder, we have 5 files excluding the `main.cpp`, `gameboard` source and header files, `mainwindow` source and header files and the header file `themes`.

GameBoard

First one implements the `GameBoard` class and its functions which are responsible for the creation, functional logic, timers, gamemodes and ending checks for the main game logic as well as the layout and UI of the game grid. Header file for this class also includes the reconfigurable static constants P , Q , K , N , M that are responsible for the probabilistic tile creation, winning number and the grid size.

MainWindow

Second one implements the `MainWindow` class and as discussed contains the methods and the code of the window of the app. It holds the Qt connections between SIGNALs and SLOTS. and is directly used by the `main.cpp`

Themes

Last one is just a header file and it includes styling choices for the tiles to be represented on the grid. It holds a single function that accepts the value of a tile as an arguments and returns the style of that tile according to the value. This header file is mainly for keeping the code clean and seperated.

6 Testing and Debugging

Some of the edge cases that are present and handled in the project are given below.

- **Full Boards:** Game ending check is handled at two steps. First the board is scanned for zeros, if they are present there is a valid move. If no zeros are present adjacent tiles are checked.

The edge case of full board is handled gracefully with the said checkpoints. If there really is no valid move, we have an initial `if` clause that checks if for the flag `isEnded` which would be raised if there are no valid moves within the function that captures the key strokes for sliding and returns empty, essentially keeping user from doing anything without resetting the game or undoing their last move. Which brings us to our second edge case.

- **Undo After Game Over:** When the game is over, the flag `isEnded` is raised and not reset until the game is restarted, but this would cause a problem if the user wants to undo their last move after the "Game Over" overlay and try again. So, we store the flag inside the `GameState` which itself gets stored in the undo stack. If the stack gets restored, the flag value gets restored as well, letting the user keep playing.

The other edge cases were handled without much problem, and they were mentioned in the previous parts of this reports.

7 Challenges and Solutions

Main hassle of this project is naturally the gameplay logic that includes the sliding and merging.

First edge case I faced on the way was the rule for a single merge for every tile on the line, meaning no cascaded merges. No cascading rule actually makes things easier since we have the option to use separate arrays. Since I initially assumed cascaded merges were allowed, I tried using a double nested for loop approach that would use a 2-pointer methods. Ultimately all of these were unnecessary after the multiple 1D arrays approach.

Another edge case that ultimately did not need any debugging was the possibility of the board getting locked in the forced move loops after a timeout in the case that there are no valid moves, the algorithm would have kept searching for a valid direction to move but fail to find any. Of course, this is not a problem since algorithm cant enter a forced move function with no valid moves.

8 Conclusion

Strongest aspect of the design is the efficient separation of concerns. Window class handles the connections and UI and the GameBoard handles the gameplay logic.

Using the correct data structures, such as 2D vectors for the grid and the undo stack (which I assume is the reason why the project does not have a undo at move 1 problem - it never existed) helped smooth development.

Possibly one of the best design choices in the code is the implementation of the line merging functions. 4 functions that are called based on the slide direction select the lines according to the slide direction, (e.g. `slideUp()` selects the lines by column, from row N - 1 to row zero) and the `mergeLine()` function always does the same thing regardless of the direction, since all the lines get passed the same.

AI Usage Statement

Gemini web page is used for debugging, helping with some of the design choices, managing the color palette of the app and general help about coding.

All the core design choices are made by me, from class hierarchy to the Doom-style coloring of the app.